



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

Факультет: «Информатика и системы управления» (ИУ)

Кафедра: «Информационная безопасность» (ИУ8)

Лабораторная работа № 1

по курсу «Математические основы искусственного
интеллекта»

Выполнил: Бородин Г.В.
Группа: ИУ8-13М
Вариант: 1

Проверил: Каганов Ю.Т.

Москва, 2025

1. Цель работы

Знакомства с основными понятиями экстремальных задач.
Исследование типов экстремумов для задач без ограничений.
Исследование типов экстремумов для задач с ограничениями.

2. Постановка задачи

Целевая функция $f(x)$, где $x = (x_1, \dots, x_n)^T$, определена на n -мерном евклидовом пространстве R^n . Ее значения характеризуют степень достижения цели, для которой поставлена или решается задача.

Множество допустимых решений $x \in X \subset R^n$ среди элементов которого осуществляется поиск.

Требуется найти такой вектор x^e , из множества допустимых решений, которому соответствует минимальное значение целевой функции на этом множестве: $f(x^e) = \min f(x)$

3. Практическая часть

3.1 Построить график поверхностей и исследовать линии уровня для функций

$$f(x_1, x_2) = 2x_1^2 - x_2^2$$

и

$$f(x_1, x_2) = 2x_1^2 - x_2^2 = \text{const} = r^2$$

при $r = 1, 2, 3, 4$.

В приложении А приведен листинг кода, с помощью которого была построена функция и линия уровня. На рисунке 1 изображен вывод программы в качестве графического окна, где изображена получившаяся поверхность и линии уровня при различных r .

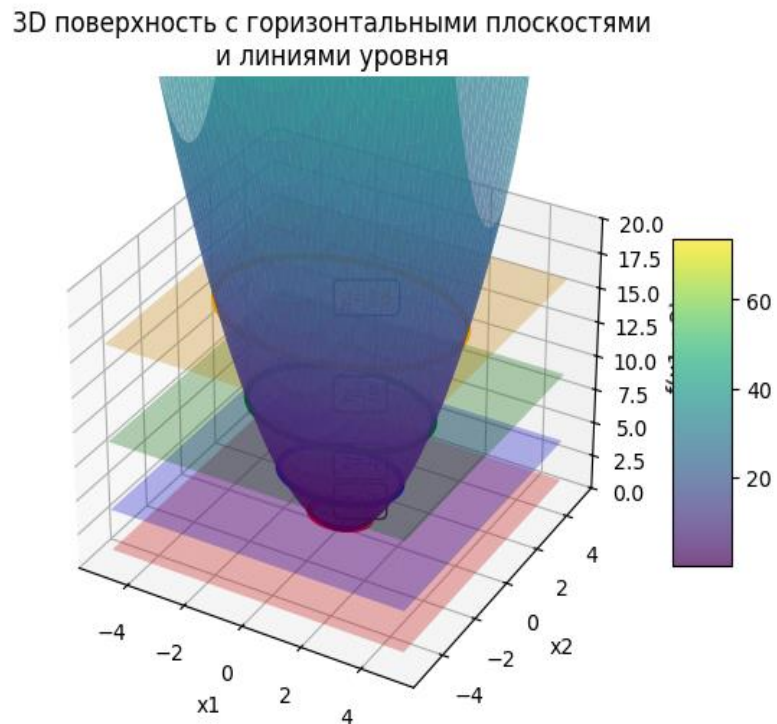


Рисунок 1 - График поверхности с линиями уровня

3.2 Исследовать типы экстремумов для многоэкстремальных задач, выявить экстремумы типа минимума и максимума, определить локальные и глобальные экстремумы.

В приложении Б приведен листинг кода, с помощью которого были определены экстремумы функции Растригина. На рисунке 2 изображена функция Растригина при заданных параметрах.

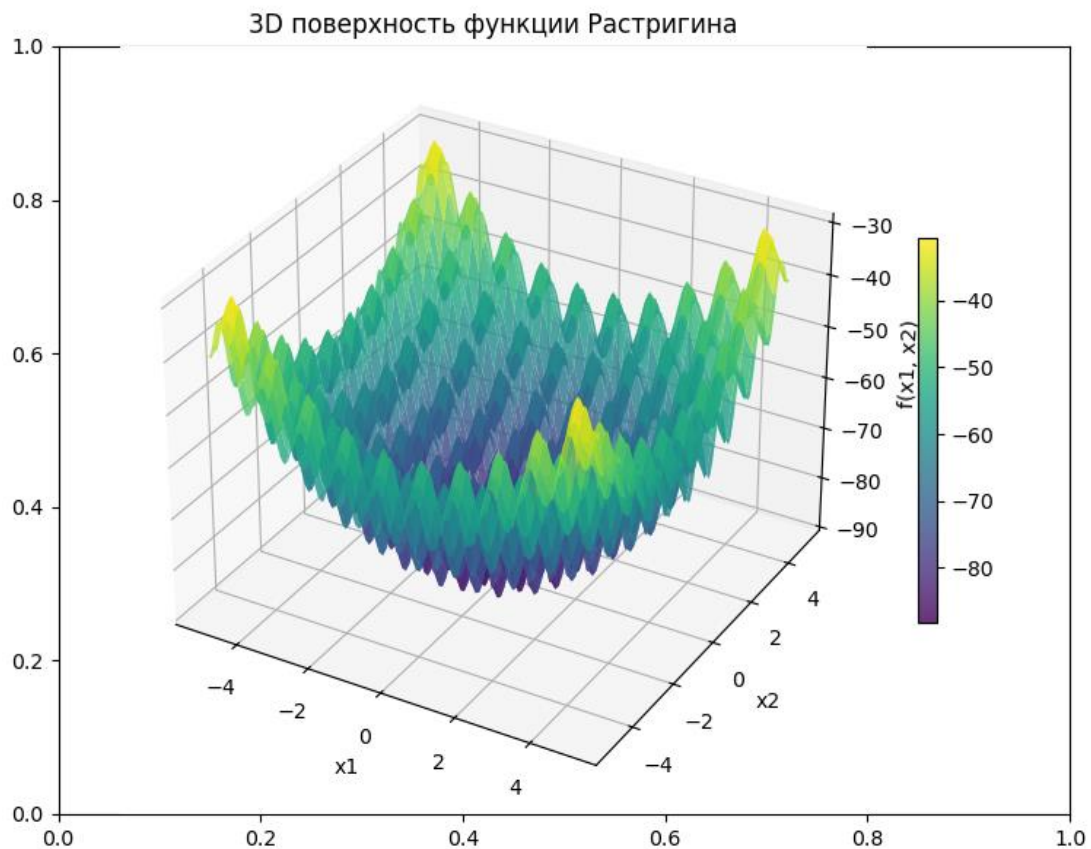
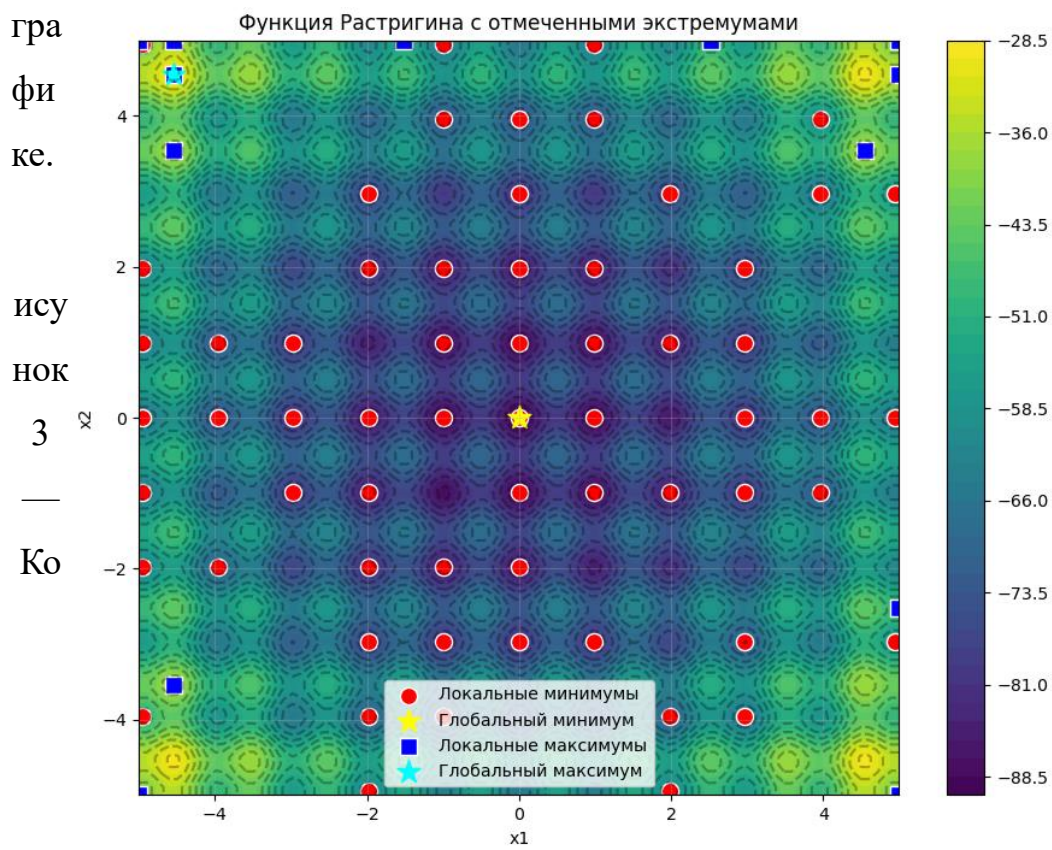


Рисунок 2 - Функция Растригина

На рисунке 3 показаны локальные и глобальные экстремумы функции Растригина с заданными параметрами на контурном



нтурный график Функции Растригина с отмеченными экстремумами

В приложении В приведен листинг кода, с помощью которого были определены экстремумы функции Стенгера с заданными параметрами. На рисунках 4 и 5 изображена функция Стенгера.

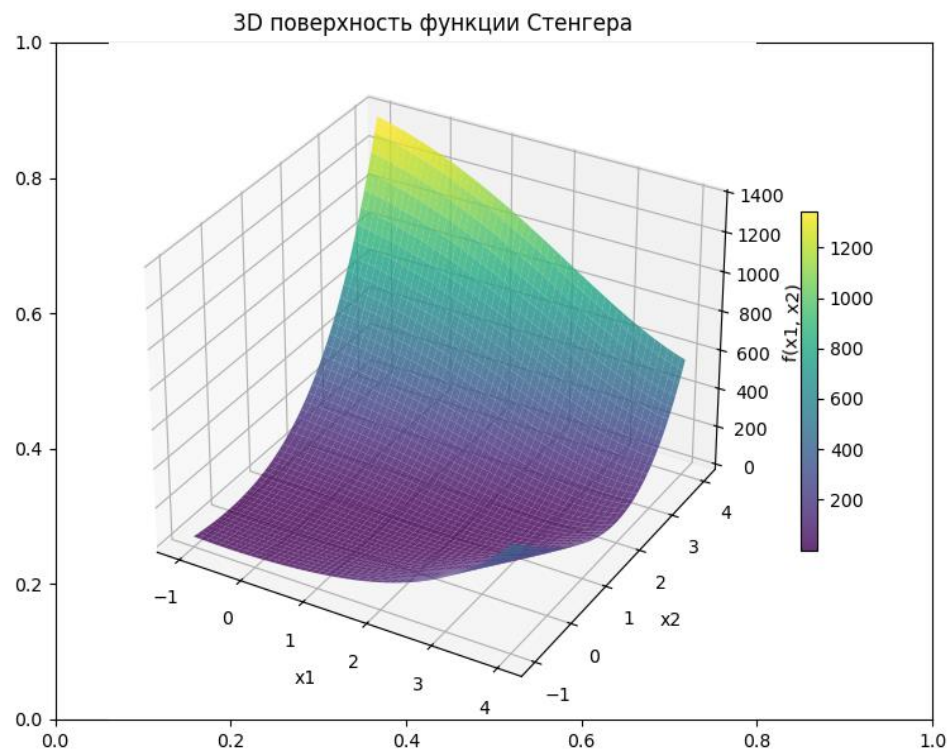


Рисунок 4 - Функция Стенгера

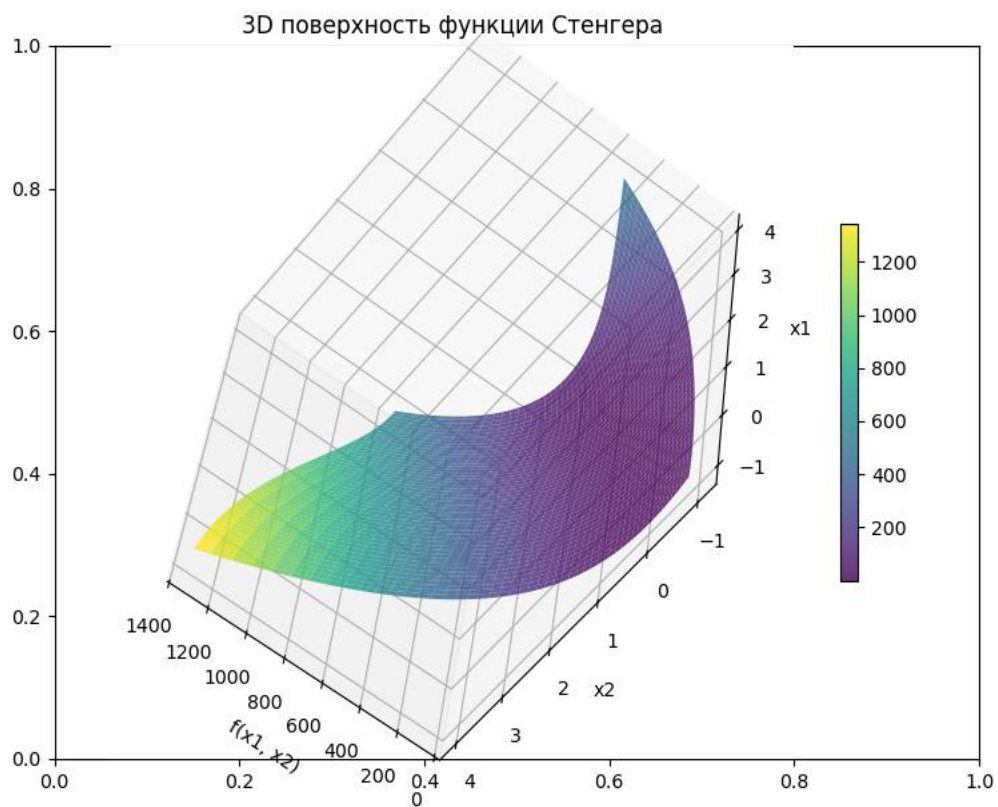


Рисунок 5 - Функция Стенгера

На рисунке 6 показаны локальные и глобальные экстремумы в области поиска на контурном графике функции Стенгера.

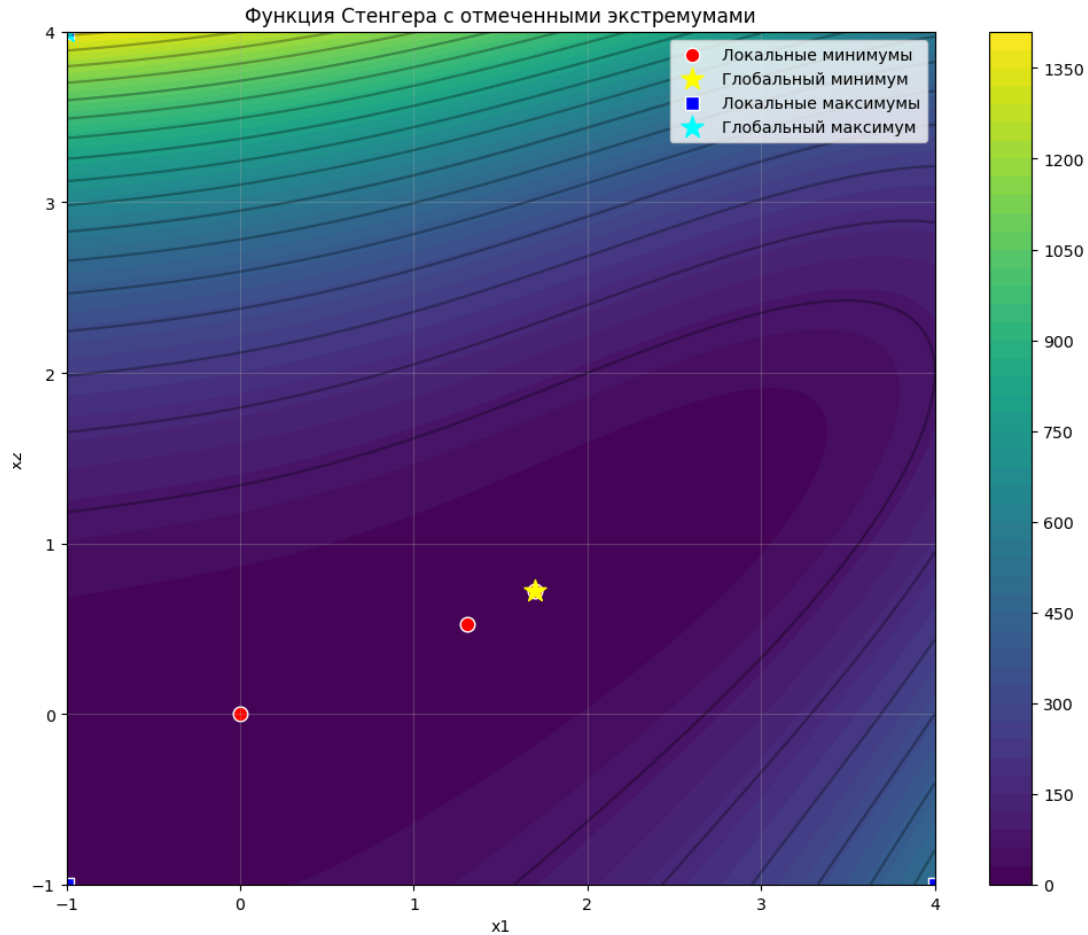


Рисунок 6 — Контурный график функции Стенгера с отмеченными экстремумами

Функция Растригина рассматривалась в области поиска $[-5, 5]$. Глобальный минимум функции совпадает с локальным минимумом в данном диапазоне и достигается при $x_1 = 0$ $x_2 = 0$. Значение функции: -90. Глобальный максимум с локальным максимумом и достигается при $x_1 = -4.5467$ $x_2 = 4.5467$. Значение функции: -29.0824.

Функция Стенгера рассматривалась в области поиска $[-1, 4]$. Были получены следующие результаты: минимум достигается при $x_1 = 1.695415$ $x_2 = 0.718608$, Значение функции: 0. Максимум достигается в точке $x_1 = -1$ $x_2 = 4$, значение функции: 1381.0

3.3 Исследовать экстремальные задачи с заданными ограничениями. Найти точки условного экстремума для целевых функций:

$$f(x_1, x_2) = x_1^2 + x_2^2$$

При ограничениях:

$$X = \{x \vee x_2^2 - x_1 + 3 = 0\}$$

И

$$f(x_1, x_2) = x_1^2 + x_2^2 - \frac{1}{2}x_1x_2$$

При ограничениях:

$$X = \{x \vee x_1 + x_2 \leq 1, x_1 \geq 0, x_2 \geq 0\}$$

В приложениях Г и Д приведен листинг кода для решения данной задачи. На рисунке 7 изображена первая заданная функция с ограничениями.

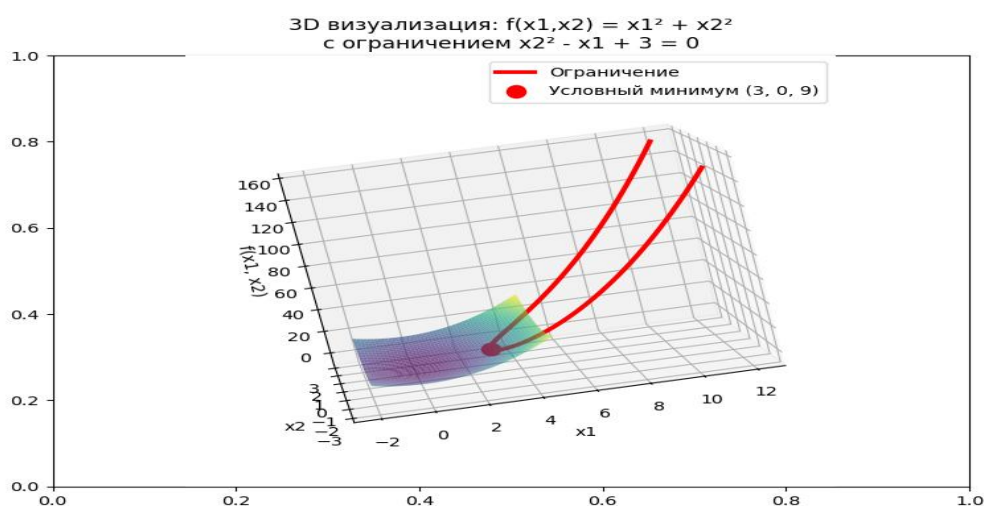


Рисунок 7 - Первая заданная функция с ограничениями

Была определена точка условного минимума с учетом ограничений: $x_1 = 3$, $x_2 = 0$. Значение функции в точке: 9. Также

была проведена проверка, что условие выполняется: Ограничение выполняется: $0 - 3 + 3 = 0$.

На рисунке 8 изображена вторая заданная функция с ограничениями и отмеченными экстремумами.

3D вид целевой функции

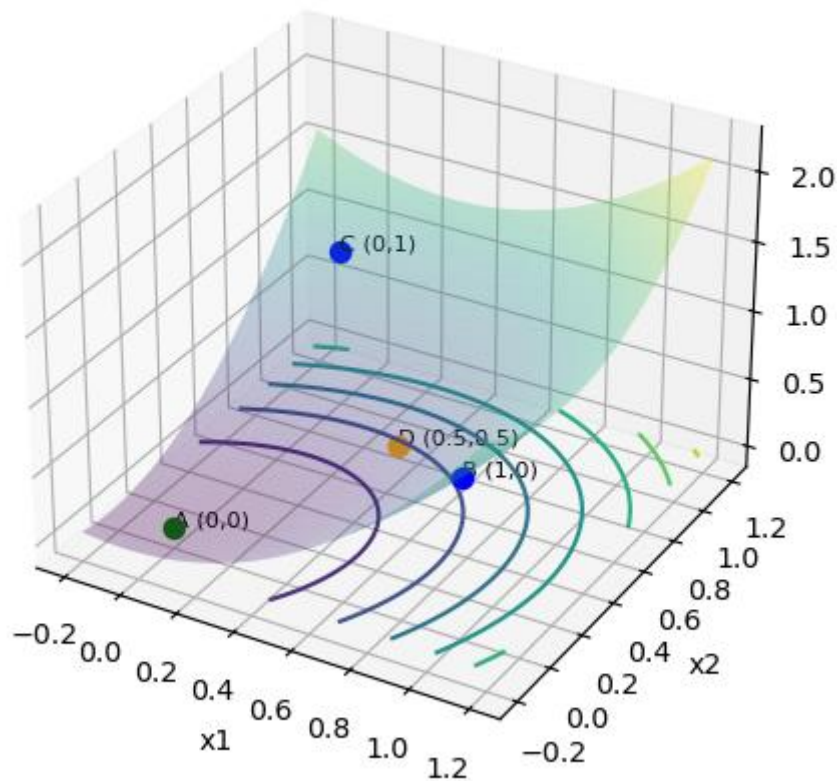


Рисунок 8 — Заданная функция с

ограничениями

На рисунке 9 изображен контурный график с отмеченными экстремумами.

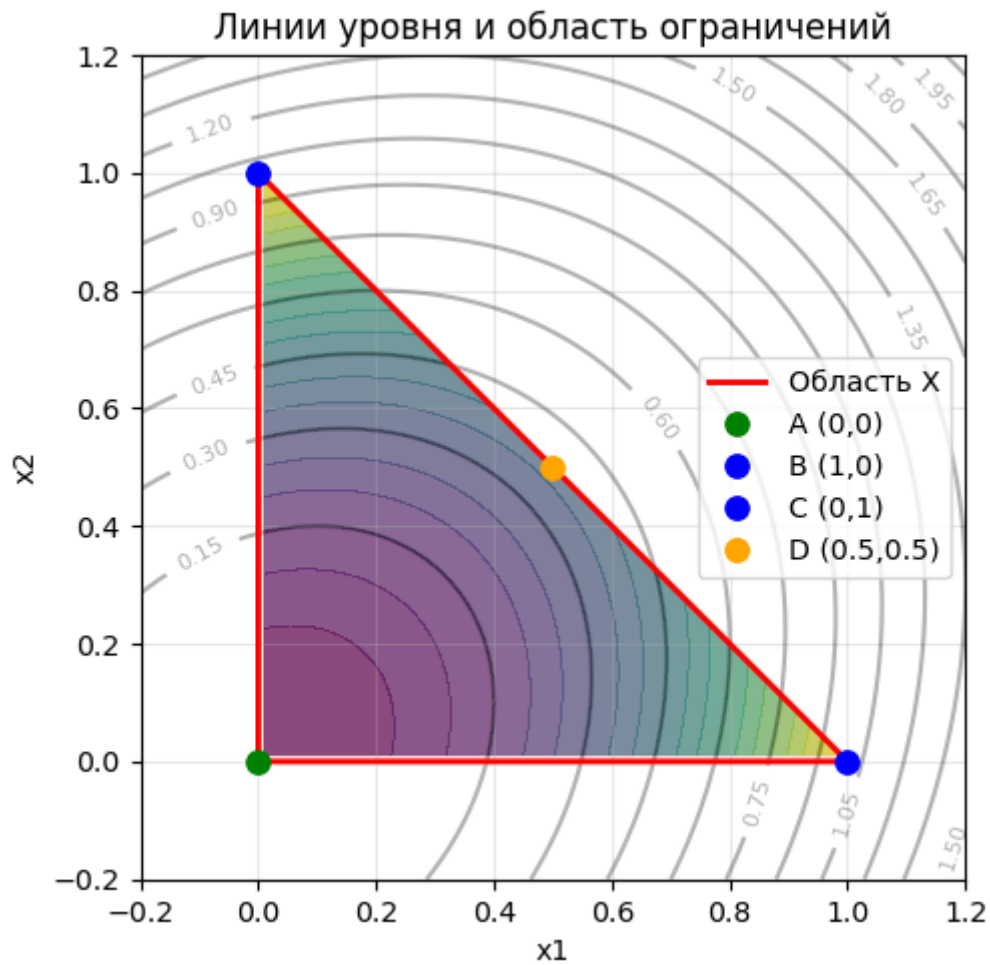


Рисунок 9 — Линии уровня заданной функции с ограничениями

Была определена точка условного минимума с учетом ограничений: $x_1 = 0$, $x_2 = 0$. Значение функции в точке: 0. Также была проведена проверка, что ограничения выполняются:

$$0 + 0 \leq 1$$

$$0 \geq 0$$

$$0 \geq 0$$

Приложение А - листинг кода для первого задания

```
import numpy as np

import matplotlib.pyplot as plt

from mpl_toolkits.mplot3d import Axes3D

x1 = np.linspace(-5, 5, 100)
x2 = np.linspace(-5, 5, 100)
X1, X2 = np.meshgrid(x1, x2)

Z = X1**2 + 2*X2**2

fig = plt.figure(figsize=(12, 9))
ax1 = fig.add_subplot(2, 2, 1, projection='3d')

surf = ax1.plot_surface(X1, X2, Z, cmap='viridis', alpha=0.7)

z_levels = [1, 4, 9, 16]
colors = ['red', 'blue', 'green', 'orange']

for z_val, color in zip(z_levels, colors):
    Z_plane = np.full_like(Z, z_val)
    plane = ax1.plot_surface(X1, X2, Z_plane, color=color, alpha=0.3)
    contour = ax1.contour(X1, X2, Z, levels=[z_val], colors=color, linewidths=3)
    ax1.text(0, 0, z_val + 0.5, f'z={z_val}', color=color, fontsize=10,
            bbox=dict(boxstyle="round,pad=0.3", facecolor='white', alpha=0.8))

ax1.set_xlabel('x1')
ax1.set_ylabel('x2')
ax1.set_zlabel('f(x1, x2)')
ax1.set_title('3D поверхность с горизонтальными плоскостями и линиями уровня')
fig.colorbar(surf, ax=ax1, shrink=0.5, aspect=5)

ax1.set_zlim(0, 20)

plt.tight_layout()
plt.show()
```

Приложение Б - листинг кода для второго задания для функции

Растригина

```

import numpy as np

import matplotlib.pyplot as plt
from scipy.optimize import minimize
import seaborn as sns

# Параметры функции
c = 5
p = 10
f0 = -100

def rastrigin(x1, x2):
    """Функция Растригина для двух переменных"""
    return (x1**2 - c * np.cos(2 * np.pi * x1) + p) + \
        (x2**2 - c * np.cos(2 * np.pi * x2) + p) + f0

# Векторизованная версия для построения графиков

def rastrigin_vectorized(X):
    x1, x2 = X
    return rastrigin(x1, x2)

# 1. Визуализация функции

def plot_rastrigin():
    x = np.linspace(-5, 5, 100)
    y = np.linspace(-5, 5, 100)
    X, Y = np.meshgrid(x, y)
    Z = rastrigin(X, Y)

    fig, axes = plt.subplots(1, 2, figsize=(15, 6))

    # 3D поверхность
    ax1 = fig.add_subplot(121, projection='3d')
    surf = ax1.plot_surface(X, Y, Z, cmap='viridis', alpha=0.8)
    ax1.set_xlabel('x1')
    ax1.set_ylabel('x2')
    ax1.set_zlabel('f(x1, x2)')
    ax1.set_title('3D поверхность функции Растригина')
    fig.colorbar(surf, ax=ax1, shrink=0.5)

    # Контурный график
    contour = axes[1].contourf(X, Y, Z, levels=50, cmap='viridis')
    axes[1].contour(X, Y, Z, levels=20, colors='black', alpha=0.3)
    axes[1].set_xlabel('x1')

```

```
axes[1].set_ylabel('x2')
axes[1].set_title('Контурный график функции Растригина')
plt.colorbar(contour, ax=axes[1])
```

```
plt.tight_layout()
plt.show()
```

```
return X, Y, Z
```

```
# 2. Поиск локальных минимумов
```

```
def find_local_minima():
    """Поиск локальных минимумов методом многозапусковой оптимизации"""
    minima = []
    minima_values = []
```

```
# Многократный запуск из случайных начальных точек
```

```
np.random.seed(42)
for _ in range(100):
    x0 = np.random.uniform(-5, 5, 2)
    result = minimize(rastrigin_vectorized, x0, bounds=[
        (-5, 5), (-5, 5)], method='L-BFGS-B')
```

```
if result.success:
    # Проверяем, не нашли ли мы уже этот минимум
    is_new = True
    for existing_min in minima:
        if np.linalg.norm(result.x - existing_min) < 0.1:
            is_new = False
            break
```

```
if is_new:
    minima.append(result.x)
    minima_values.append(result.fun)
```

```
return np.array(minima), np.array(minima_values)
```

```
# 3. Поиск локальных максимумов
```

```
def find_local_maxima():
    """Поиск локальных максимумов через минимизацию отрицательной функции"""
    maxima = []
    maxima_values = []
```

```
def negative_rastrigin(x):
    return -rastrigin(x[0], x[1])
```

```

np.random.seed(42)
for _ in range(100):
    x0 = np.random.uniform(-5, 5, 2)
    result = minimize(negative_rastrigin, x0, bounds=[
        (-5, 5), (-5, 5)], method='L-BFGS-B')

    if result.success:
        is_new = True
        for existing_max in maxima:
            if np.linalg.norm(result.x - existing_max) < 0.1:
                is_new = False
                break

        if is_new:
            maxima.append(result.x)
            maxima_values.append(-result.fun)

return np.array(maxima), np.array(maxima_values)

```

4. Анализ градиента и критических точек

```

def analyze_critical_points():
    """Аналитический анализ критических точек"""
    print("Аналитический анализ критических точек:")
    print("Функция Растргина:  $f(x_1, x_2) = (x_1^2 - 5 \cos(2\pi x_1) + 10) + (x_2^2 - 5 \cos(2\pi x_2) + 10) - 100$ ")
    print("Частные производные:")
    print(" $\partial f / \partial x_1 = 2x_1 + 10\pi \sin(2\pi x_1)$ ")
    print(" $\partial f / \partial x_2 = 2x_2 + 10\pi \sin(2\pi x_2)$ ")
    print("\nКритические точки находятся из решения уравнений:")
    print(" $2x + 10\pi \sin(2\pi x) = 0$ ")
    print("Из-за периодичности косинуса функция имеет множество локальных экстремумов")

```

Проверка некоторых ожидаемых критических точек

```

test_points = [
    [0, 0], [1, 1], [-1, -1], [2, 2], [-2, -2],
    [0.5, 0.5], [-0.5, -0.5]
]

print("\nЗначения в некоторых точках:")
for point in test_points:
    val = rastrigin(point[0], point[1])
    print(f"f({point[0]:.1f}, {point[1]:.1f}) = {val:.2f}")

```

Основной анализ

```

print("ИССЛЕДОВАНИЕ ФУНКЦИИ РАСТРИГИНА")
print("=" * 50)
print(f"Параметры: c={c}, p={p}, f0={f0}")
print("Область поиска: x1, x2 ∈ [-5, 5]")
print()

# Визуализация
X, Y, Z = plot_rastrigin()

# Аналитический анализ
analyze_critical_points()
print()

# Поиск экстремумов
local_minima, min_values = find_local_minima()
local_maxima, max_values = find_local_maxima()

print("РЕЗУЛЬТАТЫ ПОИСКА ЭКСТРЕМУМОВ:")
print("=" * 50)

# Локальные минимумы
print("\nЛОКАЛЬНЫЕ МИНИМУМЫ:")
if len(local_minima) > 0:
    global_min_idx = np.argmin(min_values)
    for i, (min_point, min_val) in enumerate(zip(local_minima, min_values)):
        marker = " □ ГЛОБАЛЬНЫЙ МИНИМУМ" if i == global_min_idx else ""
        print(
            f"Минимум {i+1}: x1={min_point[0]:.4f}, x2={min_point[1]:.4f}, f={min_val:.4f} {marker}"
        )
    else:
        print("Локальные минимумы не найдены")

# Локальные максимумы
print("\nЛОКАЛЬНЫЕ МАКСИМУМЫ:")
if len(local_maxima) > 0:
    global_max_idx = np.argmax(max_values)
    for i, (max_point, max_val) in enumerate(zip(local_maxima, max_values)):
        marker = " □ ГЛОБАЛЬНЫЙ МАКСИМУМ" if i == global_max_idx else ""
        print(
            f"Максимум {i+1}: x1={max_point[0]:.4f}, x2={max_point[1]:.4f}, f={max_val:.4f} {marker}"
        )
    else:
        print("Локальные максимумы не найдены")

# Граничные значения
print("\nГРАНИЧНЫЕ ЗНАЧЕНИЯ:")
corners = [
    [-5, -5], [-5, 5], [5, -5], [5, 5]
]
for i, corner in enumerate(corners):

```

```

val = rastrigin(corner[0], corner[1])
print(f'Угол {i+1}: x1={corner[0]}, x2={corner[1]}, f={val:.4f}')

# Сводка
print("\n" + "=" * 50)
print("СВОДКА:")
if len(min_values) > 0:
    print(f'Глобальный минимум: f = {np.min(min_values):.4f}')
if len(max_values) > 0:
    print(f'Глобальный максимум: f = {np.max(max_values):.4f}')
print(f'Диапазон значений функции: [{np.min(Z):.2f}, {np.max(Z):.2f}]')

```

Дополнительная визуализация с экстремумами

```

def plot_with_extrema(local_minima, local_maxima):
    x = np.linspace(-5, 5, 100)
    y = np.linspace(-5, 5, 100)
    X, Y = np.meshgrid(x, y)
    Z = rastrigin(X, Y)

```

```

plt.figure(figsize=(10, 8))
contour = plt.contourf(X, Y, Z, levels=50, cmap='viridis')
plt.contour(X, Y, Z, levels=20, colors='black', alpha=0.3)

```

```

# Отметить минимумы
if len(local_minima) > 0:
    plt.scatter(local_minima[:, 0], local_minima[:, 1], c='red', s=100,
                marker='o', label='Локальные минимумы', edgecolors='white')
    global_min_idx = np.argmin(min_values)
    plt.scatter(local_minima[global_min_idx, 0], local_minima[global_min_idx, 1],
                c='yellow', s=200, marker='*', label='Глобальный минимум')

```

```

# Отметить максимумы
if len(local_maxima) > 0:
    plt.scatter(local_maxima[:, 0], local_maxima[:, 1], c='blue', s=100,
                marker='s', label='Локальные максимумы', edgecolors='white')
    global_max_idx = np.argmax(max_values)
    plt.scatter(local_maxima[global_max_idx, 0], local_maxima[global_max_idx, 1],
                c='cyan', s=200, marker='*', label='Глобальный максимум')

```

```

plt.xlabel('x1')
plt.ylabel('x2')
plt.title('Функция Растригина с отмеченными экстремумами')
plt.colorbar(contour)
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

```

```
plot_with_extrema(local_minima, local_maxima)
```

Приложение В - листинг кода для второго задания для функции Стенгера

```
import numpy as np

import matplotlib.pyplot as plt
from scipy.optimize import minimize
from mpl_toolkits.mplot3d import Axes3D

def stenger(x1, x2):
    """Классическая функция Стенгера"""
    return (x1**2 - 4*x2)**2 + (x2**2 - 2*x1 + 4*x2)**2

def stenger_vectorized(X):
    x1, x2 = X
    return stenger(x1, x2)

def plot_stenger():
    x = np.linspace(-1, 4, 200)
    y = np.linspace(-1, 4, 200)
    X, Y = np.meshgrid(x, y)
    Z = stenger(X, Y)

    fig, axes = plt.subplots(1, 2, figsize=(15, 6))

    # 3D поверхность
    ax1 = fig.add_subplot(121, projection='3d')
    surf = ax1.plot_surface(X, Y, Z, cmap='viridis', alpha=0.8)
    ax1.set_xlabel('x1')
    ax1.set_ylabel('x2')
    ax1.set_zlabel('f(x1, x2)')
    ax1.set_title('3D поверхность функции Стенгера')
    fig.colorbar(surf, ax=ax1, shrink=0.5)

    # Контурный график
    contour = axes[1].contourf(X, Y, Z, levels=50, cmap='viridis')
    axes[1].contour(X, Y, Z, levels=20, colors='black', alpha=0.3)
    axes[1].set_xlabel('x1')
    axes[1].set_ylabel('x2')
    axes[1].set_title('Контурный график функции Стенгера')
    plt.colorbar(contour, ax=axes[1])

    plt.tight_layout()
    plt.show()
```

```
return X, Y, Z
```

```
def find_extrema(num_points=500, tol=1e-5):  
    """Поиск экстремумов с улучшенным алгоритмом"""
```

```
def find_minima():  
    minima = []  
    minima_values = []
```

```
np.random.seed(42)  
for _ in range(num_points):  
    x0 = np.random.uniform(-1, 4, 2)  
    result = minimize(stenger_vectorized, x0, bounds=[(-1, 4), (-1, 4)],  
method='L-BFGS-B', tol=1e-10)
```

```
if result.success:  
    # Проверяем, находится ли точка в области  
    if (-1 <= result.x[0] <= 4) and (-1 <= result.x[1] <= 4):  
        # Более строгая проверка уникальности  
        is_new = True  
        for existing_min in minima:  
            if np.linalg.norm(result.x - existing_min) < tol:  
                is_new = False  
                break
```

```
if is_new:  
    minima.append(result.x)  
    minima_values.append(result.fun)
```

```
return np.array(minima), np.array(minima_values)
```

```
def find_maxima():  
    maxima = []  
    maxima_values = []
```

```
def negative_stenger(x):  
    return -stenger(x[0], x[1])
```

```
np.random.seed(42)  
for _ in range(num_points):  
    x0 = np.random.uniform(-1, 4, 2)  
    result = minimize(negative_stenger, x0, bounds=[(-1, 4), (-1, 4)],  
method='L-BFGS-B', tol=1e-10)
```

```
if result.success:  
    if (-1 <= result.x[0] <= 4) and (-1 <= result.x[1] <= 4):  
        is_new = True
```

```

for existing_max in maxima:
    if np.linalg.norm(result.x - existing_max) < tol:
        is_new = False
        break

    if is_new:
        maxima.append(result.x)
        maxima_values.append(-result.fun)

return np.array(maxima), np.array(maxima_values)

return find_minima(), find_maxima()

# Запуск анализа
print("ИССЛЕДОВАНИЕ ФУНКЦИИ СТЕНГЕРА")
print("=" * 50)
print("Область поиска: x1, x2 ∈ [-1, 4]")
print()

# Визуализация
X, Y, Z = plot_stenger()

# Поиск экстремумов
(local_minima, min_values), (local_maxima, max_values) = find_extrema()

print("РЕЗУЛЬТАТЫ ПОИСКА ЭКСТРЕМУМОВ:")
print("=" * 50)

# Локальные минимумы
print("\nЛОКАЛЬНЫЕ МИНИМУМЫ:")
if len(local_minima) > 0:
    global_min_idx = np.argmin(min_values)
    for i, (min_point, min_val) in enumerate(zip(local_minima, min_values)):
        marker = " □ ГЛОБАЛЬНЫЙ МИНИМУМ" if i == global_min_idx else ""
        print(
            f"Минимум {i+1}: x1={min_point[0]:.6f}, x2={min_point[1]:.6f}, f={min_val:.6f} {marker}"
        )
    else:
        print("Локальные минимумы не найдены")

# Локальные максимумы
print("\nЛОКАЛЬНЫЕ МАКСИМУМЫ:")
if len(local_maxima) > 0:
    global_max_idx = np.argmax(max_values)
    for i, (max_point, max_val) in enumerate(zip(local_maxima, max_values)):
        marker = " □ ГЛОБАЛЬНЫЙ МАКСИМУМ" if i == global_max_idx else ""
        print(
            f"Максимум {i+1}: x1={max_point[0]:.6f}, x2={max_point[1]:.6f}, f={max_val:.6f} {marker}"
        )

```

```

else:
    print("Локальные максимумы не найдены")

# Анализ критических точек
print("\nАНАЛИТИЧЕСКИЙ АНАЛИЗ:")
print("Функция Стенгера имеет множество локальных минимумов")
print("Известные особенности:")
print("- Невыпуклая функция")
print("- Множество локальных минимумов")
print("- Сложная поверхность с 'оврагами'")

# Визуализация с экстремумами

def plot_with_extrema(local_minima, local_maxima):
    x = np.linspace(-1, 4, 200)
    y = np.linspace(-1, 4, 200)
    X, Y = np.meshgrid(x, y)
    Z = stenger(X, Y)

    plt.figure(figsize=(12, 10))
    contour = plt.contourf(X, Y, Z, levels=50, cmap='viridis')
    plt.contour(X, Y, Z, levels=20, colors='black', alpha=0.3)

    if len(local_minima) > 0:
        plt.scatter(local_minima[:, 0], local_minima[:, 1], c='red', s=80,
                    marker='o', label='Локальные минимумы', edgecolors='white')
        global_min_idx = np.argmin(min_values)
        plt.scatter(local_minima[global_min_idx, 0], local_minima[global_min_idx, 1],
                    c='yellow', s=200, marker='*', label='Глобальный минимум')

    if len(local_maxima) > 0:
        plt.scatter(local_maxima[:, 0], local_maxima[:, 1], c='blue', s=80,
                    marker='s', label='Локальные максимумы', edgecolors='white')
        global_max_idx = np.argmax(max_values)
        plt.scatter(local_maxima[global_max_idx, 0], local_maxima[global_max_idx, 1],
                    c='cyan', s=200, marker='*', label='Глобальный максимум')

    plt.xlabel('x1')
    plt.ylabel('x2')
    plt.title('Функция Стенгера с отмеченными экстремумами')
    plt.colorbar(contour)
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.show()

plot_with_extrema(local_minima, local_maxima)

```

Приложение Г — листинг кода для третьего задания для первой функции.

```
import numpy as np

import matplotlib.pyplot as plt
from scipy.optimize import minimize
from mpl_toolkits.mplot3d import Axes3D

def objective_function(x):
    """Целевая функция:  $f(x_1, x_2) = x_1^2 + x_2^2$ """
    return x[0]**2 + x[1]**2

def constraint(x):
    """Ограничение:  $x_2^2 - x_1 + 3 = 0$ """
    return x[1]**2 - x[0] + 3

def solve_constrained_optimization():
    """Решение задачи условной оптимизации"""

    # Определяем ограничение
    constraints = {'type': 'eq', 'fun': constraint}

    # Начальные точки для поиска
    initial_points = [
        [0, 0], [1, 1], [-1, -1], [2, 2], [-2, -2],
        [3, 0], [0, 2], [0, -2]
    ]

    results = []
```

```

print("ЧИСЛЕННОЕ РЕШЕНИЕ")
print("=" * 50)

for i, x0 in enumerate(initial_points):
    result = minimize(objective_function, x0,
                      constraints=constraints,
                      method='SLSQP',
                      options={'ftol': 1e-10})

    if result.success:
        # Проверяем уникальность точки
        is_new = True
        for existing in results:
            if np.linalg.norm(result.x - existing['point']) < 1e-6:
                is_new = False
                break

        if is_new:
            results.append({
                'point': result.x,
                'value': result.fun,
                'initial': x0
            })
        print(
            f"Начальная точка {x0}: найдена точка ({result.x[0]:.6f}, {result.x[1]:.6f}), f = {result.fun:.6f}")

    return results

def plot_solution():
    """Визуализация решения"""

    # Создаем сетку для поверхности
    x1 = np.linspace(-2, 5, 100)
    x2 = np.linspace(-3, 3, 100)
    X1, X2 = np.meshgrid(x1, x2)
    Z = X1**2 + X2**2

    # Ограничение: парабола  $x_1 = x_2^2 + 3$ 
    constraint_x2 = np.linspace(-3, 3, 100)
    constraint_x1 = constraint_x2**2 + 3

    # Значения функции на ограничении
    constraint_z = constraint_x1**2 + constraint_x2**2

    fig, axes = plt.subplots(1, 2, figsize=(15, 6))

```

```

# 3D график
ax1 = fig.add_subplot(121, projection='3d')
surf = ax1.plot_surface(X1, X2, Z, cmap='viridis', alpha=0.7)
ax1.plot(constraint_x1, constraint_x2, constraint_z,
'r-', linewidth=3, label='Ограничение')
ax1.scatter(3, 0, 9, color='red', s=100,
label='Условный минимум (3, 0, 9)')
ax1.set_xlabel('x1')
ax1.set_ylabel('x2')
ax1.set_zlabel('f(x1, x2)')
ax1.set_title(
'3D визуализация:  $f(x_1, x_2) = x_1^2 + x_2^2$  с ограничением  $x_2^2 - x_1 + 3 = 0$ ')
ax1.legend()

```

```

# Контурный график
contour = axes[1].contour(X1, X2, Z, levels=20, cmap='viridis')
axes[1].plot(constraint_x1, constraint_x2, 'r-',
linewidth=2, label='Ограничение')
axes[1].scatter(3, 0, color='red', s=100, label='Условный минимум (3, 0)')
axes[1].set_xlabel('x1')
axes[1].set_ylabel('x2')
axes[1].set_title('Контурный график с ограничением')
axes[1].legend()
axes[1].grid(True, alpha=0.3)
plt.colorbar(contour, ax=axes[1])

```

```

plt.tight_layout()
plt.show()

```

```

# Запуск анализа
print("ИССЛЕДОВАНИЕ УСЛОВНОГО ЭКСТРЕМУМА")
print("=" * 60)
print("Целевая функция:  $f(x_1, x_2) = x_1^2 + x_2^2$ ")
print("Ограничение:  $x_2^2 - x_1 + 3 = 0$ ")
print()

```

```

# Аналитическое решение
print("АНАЛИТИЧЕСКОЕ РЕШЕНИЕ:")
print("=" * 40)
print("Метод подстановки:")
print(" $x_1 = x_2^2 + 3$ ")
print(" $f(x_2) = (x_2^2 + 3)^2 + x_2^2 = x_2^4 + 7x_2^2 + 9$ ")
print(" $f'(x_2) = 4x_2^3 + 14x_2 = 2x_2(2x_2^2 + 7)$ ")
print("Критические точки:  $x_2 = 0$ ")
print(" $x_1 = 0^2 + 3 = 3$ ")
print(" $f''(0) = 14 > 0$  □ МИНИМУМ")

```

```

print("Точка условного минимума: (3, 0)")
print("Значение функции: f(3, 0) = 9")
print()

print("Метод множителей Лагранжа:")
print("L(x1, x2, λ) = x12 + x22 + λ(x22 - x1 + 3)")
print("∂L/∂x1 = 2x1 - λ = 0 ⇔ λ = 2x1")
print("∂L/∂x2 = 2x2 + 2λx2 = 0 ⇔ x2(1 + λ) = 0")
print("∂L/∂λ = x22 - x1 + 3 = 0")
print("Решение: x2 = 0, x1 = 3, λ = 6")
print()

# Численное решение
results = solve_constrained_optimization()

print("\n" + "=" * 60)
print("ИТОГОВЫЕ РЕЗУЛЬТАТЫ:")
print("=" * 60)
if results:
    best_result = min(results, key=lambda x: x['value'])
    print(f"⇔ УСЛОВНЫЙ МИНИМУМ:")
    print(
        f' Точка: ({best_result["point"][0]:.8f}, {best_result["point"][1]:.8f})'
    )
    print(f' Значение функции: {best_result["value"]:.8f}')
    print(f' Проверка ограничения: {constraint(best_result["point"]):.2e}')
else:
    print("Решения не найдены")

# Визуализация
plot_solution()

```

Приложение Д — листинг кода для третьего задания для первой функции.

```

import numpy as np

import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# 1. Определяем целевую функцию и ограничения

def f(x1, x2):
    return x1**2 + x2**2 - 0.5*x1*x2

```

```
def constraint(x1, x2):
    """Проверка ограничений:  $x1 + x2 \leq 1$ ,  $x1 \geq 0$ ,  $x2 \geq 0$ """
    return (x1 + x2 <= 1) & (x1 >= 0) & (x2 >= 0)
```

```
# 2. Создаем сетку для визуализации
x1 = np.linspace(-0.2, 1.2, 100)
x2 = np.linspace(-0.2, 1.2, 100)
X1, X2 = np.meshgrid(x1, x2)
Z = f(X1, X2)
```

```
# Маска для допустимой области
mask = constraint(X1, X2)
Z_masked = np.where(mask, Z, np.nan)
```

```
# 3. Кандидаты в точки экстремума
candidates = [
    (0, 0, 'A (0,0)'), # вершина
    (1, 0, 'B (1,0)'), # вершина
    (0, 1, 'C (0,1)'), # вершина
    (0.5, 0.5, 'D (0.5,0.5)') # точка на гипотенузе
]
```

```
# 4. Вычисляем значения функции в кандидатах
print("Точки кандидаты в экстремумы:")
print("=" * 40)
for x1_val, x2_val, label in candidates:
    value = f(x1_val, x2_val)
    print(f'{label}: f = {value:.3f}')
```

```
# 5. Визуализация
fig = plt.figure(figsize=(15, 5))
```

```
# 5.1 2D вид с линиями уровня
ax1 = plt.subplot(1, 3, 1)
contour = ax1.contour(X1, X2, Z, levels=20, colors='black', alpha=0.3)
ax1.contourf(X1, X2, Z_masked, levels=20, alpha=0.7)
ax1.clabel(contour, inline=True, fontsize=8)
```

```
# Рисуем треугольник ограничений
triangle_x = [0, 1, 0, 0]
triangle_y = [0, 0, 1, 0]
ax1.plot(triangle_x, triangle_y, 'r-', linewidth=2, label='Область X')
ax1.fill(triangle_x, triangle_y, 'red', alpha=0.1)
```

```
# Отмечаем кандидатов
colors = ['green', 'blue', 'blue', 'orange']
for i, (x1_val, x2_val, label) in enumerate(candidates):
```

```
ax1.plot(x1_val, x2_val, 'o', color=colors[i], markersize=8, label=label)
```

```
ax1.set_xlabel('x1')
ax1.set_ylabel('x2')
ax1.set_title('Линии уровня и область ограничений')
ax1.legend()
ax1.grid(True, alpha=0.3)
ax1.set_aspect('equal')
```

```
# 5.2 3D поверхность
```

```
ax2 = plt.subplot(1, 3, 2, projection='3d')
surf = ax2.plot_surface(X1, X2, Z, alpha=0.3, cmap='viridis')
ax2.contour(X1, X2, Z, zdir='z', offset=Z.min(), cmap='viridis')
```

```
# Отмечаем кандидатов на 3D графике
```

```
for i, (x1_val, x2_val, label) in enumerate(candidates):
    z_val = f(x1_val, x2_val)
    ax2.scatter(x1_val, x2_val, z_val, color=colors[i], s=50, label=label)
    ax2.text(x1_val, x2_val, z_val, label, fontsize=8)
```

```
ax2.set_xlabel('x1')
ax2.set_ylabel('x2')
ax2.set_zlabel('f(x1,x2)')
ax2.set_title('3D вид целевой функции')
```

```
# 5.3 Значения функции вдоль границы  $x_1 + x_2 = 1$ 
```

```
ax3 = plt.subplot(1, 3, 3)
t = np.linspace(0, 1, 100)
x1_boundary = t
x2_boundary = 1 - t
f_boundary = f(x1_boundary, x2_boundary)
```

```
ax3.plot(t, f_boundary, 'b-', linewidth=2, label='f вдоль  $x_1+x_2=1$ ')
ax3.axvline(x=0.5, color='orange', linestyle='--',
            alpha=0.7, label='x1=0.5 (минимум)')
ax3.plot(0, f(0, 1), 'bo', markersize=8, label='(0,1)')
ax3.plot(1, f(1, 0), 'bo', markersize=8, label='(1,0)')
ax3.plot(0.5, f(0.5, 0.5), 'ro', markersize=8, label='(0.5,0.5)')
```

```
ax3.set_xlabel('x1 (x2 = 1 - x1)')
ax3.set_ylabel('f(x1, x2)')
ax3.set_title('Функция вдоль границы  $x_1+x_2=1$ ')
ax3.legend()
ax3.grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.show()
```

```
# 6. Вывод результатов
```

```
print("\n" + "=" * 50)
```

```
print("РЕЗУЛЬТАТЫ:")
```

```
print("=" * 50)
```

```
print("Глобальный минимум: (0, 0), f = 0")
```

```
print("Глобальный максимум: (1, 0) и (0, 1), f = 1")
```

```
print("Локальный минимум на границе: (0.5, 0.5), f = 0.375")
```